

**Universidade do Minho**  
Escola de Engenharia  
Licenciatura em Engenharia Informática

# **Dados e Aprendizagem Automática**

## **Trabalho Prático**

Grupo 7

|                       |                       |                       |                   |
|-----------------------|-----------------------|-----------------------|-------------------|
| <b>Edgar Ferreira</b> | <b>Fernando Pires</b> | <b>Pedro Teixeira</b> | <b>Sara Silva</b> |
| pg60250               | pg60253               | pg60294               | pg61546           |

|                 |  |
|-----------------|--|
| Data da Receção |  |
| Responsável     |  |
| Avaliação       |  |
| Observações     |  |

## Dados e Aprendizagem Automática

**Edgar Ferreira**  
pg60250

**Fernando Pires**  
pg60253

**Pedro Teixeira**  
pg60294

**Sara Silva**  
pg61546

janeiro, 2026

# Índice

|                                              |           |
|----------------------------------------------|-----------|
| <b>1. Introdução</b>                         | <b>1</b>  |
| <b>2. Metodologia</b>                        | <b>2</b>  |
| <b>3. <i>Business Understanding</i></b>      | <b>3</b>  |
| <b>4. <i>Data Understanding</i></b>          | <b>4</b>  |
| <b>5. <i>Data Preparation</i></b>            | <b>6</b>  |
| 5.1. Primeira Abordagem                      | 6         |
| 5.2. Segunda Abordagem                       | 6         |
| 5.3. Terceira Abordagem                      | 7         |
| 5.4. Quarta Abordagem                        | 7         |
| 5.5. Quinta Abordagem                        | 7         |
| <b>6. <i>Modeling and Evaluation</i></b>     | <b>8</b>  |
| 6.1. <i>Decision Trees</i>                   | 8         |
| 6.2. <i>Random Forest</i>                    | 8         |
| 6.3. XGBoost                                 | 9         |
| 6.4. kNN                                     | 9         |
| 6.5. <i>Stacking</i>                         | 9         |
| 6.6. <i>Neural Networks (MLP Classifier)</i> | 10        |
| 6.7. <i>Naive Bayes</i>                      | 10        |
| 6.8. Comparação de Resultados entre Modelos  | 10        |
| <b>7. Conclusões</b>                         | <b>12</b> |
| <b>8. Avaliação por Pares</b>                | <b>13</b> |

## Lista de Figuras

|           |                                                                                                        |    |
|-----------|--------------------------------------------------------------------------------------------------------|----|
| Figura 1  | Modelo CRISP-DM                                                                                        | 2  |
| Figura 2  | <i>Missing Values dataset</i> de Teste                                                                 | 4  |
| Figura 3  | <i>Missing Values dataset</i> de Treino                                                                | 4  |
| Figura 4  | <i>Dataset</i> apenas para a cidade do Porto                                                           | 4  |
| Figura 5  | Valores da <code>average_precipitation</code>                                                          | 4  |
| Figura 6  | Distribuição dos tipos de chuva no <i>dataset</i> de treino ( <code>average_rain</code> )              | 5  |
| Figura 7  | Distribuição dos tipos de nebulosidade no <i>dataset</i> de treino ( <code>average_cloudiness</code> ) | 5  |
| Figura 8  | Melhores resultados do <i>Decision Trees</i> na plataforma <i>Kaggle</i>                               | 8  |
| Figura 9  | Melhores resultados do <i>Random Forest</i> na plataforma <i>Kaggle</i>                                | 9  |
| Figura 10 | Melhores resultados do XGBoost na plataforma <i>Kaggle</i>                                             | 9  |
| Figura 11 | Melhores resultados do kNN na plataforma <i>Kaggle</i>                                                 | 9  |
| Figura 12 | Melhores resultados do <i>Stacking</i> na plataforma <i>Kaggle</i>                                     | 10 |
| Figura 13 | Melhores resultados das <i>Neural Networks</i> na plataforma <i>Kaggle</i>                             | 10 |
| Figura 14 | Melhores resultados do <i>Naive Bayes</i> na plataforma <i>Kaggle</i>                                  | 10 |

# 1. Introdução

A modelação e previsão do tráfego rodoviário constituem um problema relevante em diversos domínios, nomeadamente no planeamento urbano, na gestão de infraestruturas e na otimização da mobilidade. Este problema caracteriza-se por uma elevada complexidade, resultante do seu comportamento estocástico e não linear, influenciado por múltiplos fatores temporais, espaciais e contextuais. Neste contexto, os métodos de **Machine Learning** têm demonstrado um grande potencial para capturar padrões complexos nos dados e produzir previsões mais precisas do que abordagens tradicionais.

O presente trabalho prático, desenvolvido no âmbito da unidade curricular de Dados e Aprendizagem Automática, tem como principal objetivo o desenvolvimento, otimização e avaliação de modelos de *Machine Learning* para a previsão do fluxo de tráfego rodoviário numa cidade portuguesa, com base num conjunto de dados recolhidos ao longo de mais de um ano. Para tal, recorre-se a um *dataset* disponibilizado no contexto de uma competição na plataforma **Kaggle**, o qual contém informação relevante sobre o volume de tráfego registado em diferentes períodos temporais.

O trabalho encontra-se estruturado em duas componentes principais. A primeira corresponde a uma tarefa de análise e validação, onde se procede à exploração, preparação e compreensão do *dataset*, bem como à avaliação crítica dos resultados obtidos com diferentes modelos. A segunda componente assume a forma de uma tarefa de competição, cujo objetivo é maximizar o desempenho preditivo dos modelos desenvolvidos, medido através de uma métrica de *accuracy* definida no contexto da competição. Ao longo do relatório, são descritas as metodologias adotadas, os modelos implementados, o processo de afinação de hiperparâmetros e a análise crítica dos resultados, procurando sempre interpretar a sua relevância e utilidade no contexto do problema em estudo.

## 2. Metodologia

Para a realização deste trabalho prático, adotámos a metodologia **CRISP-DM** (*Cross Industry Standard Process for Data Mining*), uma abordagem amplamente utilizada para projetos de mineração de dados.

Seguindo a sua estrutura, iniciámos pelo **entendimento** do negócio, definindo os objetivos a alcançar com o modelo a desenvolver. Posteriormente, realizámos a **compreensão** dos dados, garantindo um conhecimento aprofundado da sua estrutura e características antes de avançarmos para a sua **preparação**. Nesta etapa, efetuámos transformações e limpezas necessárias para assegurar a qualidade dos dados antes da **modelação**.

Na fase seguinte, aplicámos diferentes técnicas de modelação ao problema em análise, seguindo-se a **avaliação** dos modelos gerados para determinar a sua eficácia e adequação ao objetivo definido. A última fase do **CRISP-DM** corresponde à **implementação**, que, no contexto deste trabalho, não foi abordada.

É importante destacar que, para melhorar a qualidade do projeto, sempre que necessário, revisitámos etapas anteriores, ajustando abordagens e refinando decisões com base nos resultados obtidos.

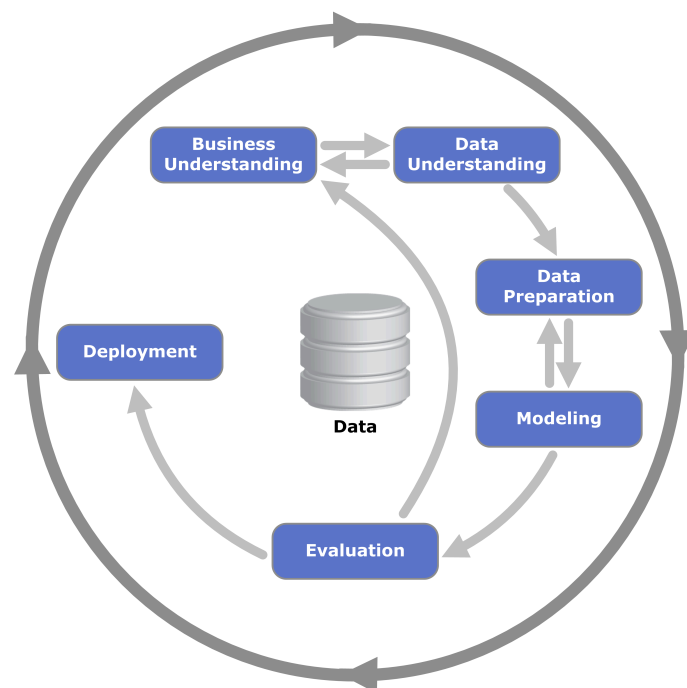


Figura 1: Modelo CRISP-DM

### 3. Business Understanding

Foi-nos atribuído pela equipa docente um conjunto de dados relacionado com mobilidade urbana e tráfego rodoviário, composto por dois ficheiros: `training_data.csv` e `test_data.csv`. O primeiro é utilizado para o desenvolvimento e treino dos modelos de *Machine Learning*, contendo a variável alvo, enquanto o segundo é usado para avaliar o desempenho dos modelos em dados não vistos, não incluindo a variável a prever.

Os atributos presentes nos *datasets* são os seguintes:

`city_name` - nome da cidade em causa;

`record_date` - o *timestamp* associado ao registo;

`average_speed_diff` - a diferença de velocidade corresponde à diferença entre (1.) a velocidade máxima que os carros podem atingir em cenários sem trânsito e (2.) a velocidade que realmente se verifica. Quanto mais alto o valor, maior é a diferença entre o que se está a andar no momento e o que se deveria estar a andar sem trânsito, i.e., valores altos deste atributo implicam que se está a andar mais devagar;

`average_free_flow_speed` - o valor médio da velocidade máxima que os carros podem atingir em cenários sem trânsito;

`average_time_diff` - o valor médio da diferença do tempo que se demora a percorrer um determinado conjunto de ruas. Quanto mais alto o valor, maior é a diferença entre o tempo que demora para se percorrer as ruas e o que se deveria demorar sem trânsito, i.e., valores altos implicam que se está a demorar mais tempo a atravessar o conjunto de ruas;

`average_free_flow_time` - o valor médio do tempo que demora a percorrer um determinado conjunto de ruas quando não há trânsito;

`luminosity` - o nível de luminosidade que se verificava na cidade do Porto;

`average_temperature` - valor médio da temperatura para o `record_date` na cidade do Porto;

`average_atmosp_pressure` - valor médio da pressão atmosférica para o `record_date` na cidade do Porto;

`average_humidity` - valor médio de humidade para o `record_date` na cidade do Porto;

`average_wind_speed` - valor médio da velocidade do vento para o `record_date` na cidade do Porto;

`average_cloudiness` - o valor médio da percentagem de nuvens para o `record_date` na cidade do Porto;

`average_precipitation` - valor médio de precipitação para o `record_date` na cidade do Porto;

`average_rain` - avaliação qualitativa do nível de precipitação para o `record_date` na cidade do Porto.

Entre os atributos descritos, destaca-se a variável `average_speed_diff`, que constitui a **variável alvo** do problema preditivo. Esta variável encontra-se disponível apenas no ficheiro `training_data.csv`, sendo utilizada no processo de treino e validação dos modelos. O objetivo deste trabalho é **prever o seu valor** para os registos presentes no ficheiro `test_data.csv`.

A diversidade de variáveis temporais, meteorológicas e de tráfego fornece uma base sólida para a construção de modelos preditivos robustos, permitindo analisar o impacto de diferentes condições no nível de congestionamento urbano.

## 4. Data Understanding

A exploração inicial dos *datasets* de treino e de teste permitiu uma melhor compreensão da estrutura dos dados e das principais características que irão influenciar as próximas etapas do projeto. Os *datasets* apresentam medições de tráfego urbano e condições meteorológicas registadas ao longo do tempo, mas foi possível identificar discrepâncias importantes entre eles, não apenas na quantidade de registos, mas também na disponibilidade de determinados atributos e na presença de valores em falta.

No *dataset* de treino, composto por 6812 registos, identificou-se a existência de **missing values** em vários atributos, com destaque para o campo `average_rain`, onde se verificam 6249 valores em falta, correspondendo a quase a totalidade dos registos (92%). No *dataset* de teste (com 1500 linhas) foi identificada uma situação parecida, na qual `average_rain` apresenta 1360 valores em falta (91%).

```
df_test.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 13 columns):
#   Column                      Non-Null Count  Dtype
---  ---                      ---
0   city_name                   1500 non-null   object
1   record_date                 1500 non-null   object
2   AVERAGE_FREE_FLOW_SPEED    1500 non-null   float64
3   AVERAGE_TIME_DIFF          1500 non-null   float64
4   AVERAGE_FREE_FLOW_TIME     1500 non-null   float64
5   LUMINOSITY                  1500 non-null   object
6   AVERAGE_TEMPERATURE        1500 non-null   float64
7   AVERAGE_ATMOSP_PRESSURE     1500 non-null   float64
8   AVERAGE_HUMIDITY           1500 non-null   float64
9   AVERAGE_WIND_SPEED         1500 non-null   float64
10  AVERAGE_CLOUDINESS         901 non-null    object
11  AVERAGE_PRECIPITATION       1500 non-null   float64
12  AVERAGE_RAIN               140 non-null    object
dtypes: float64(8), object(5)
memory usage: 152.5+ KB
```

Figura 2: Missing Values dataset de Teste

```
df_train.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6812 entries, 0 to 6811
Data columns (total 14 columns):
#   Column                      Non-Null Count  Dtype
---  ---                      ---
0   city_name                   6812 non-null   object
1   record_date                 6812 non-null   object
2   AVERAGE_SPEED_DIFF         4612 non-null   object
3   AVERAGE_FREE_FLOW_SPEED    6812 non-null   float64
4   AVERAGE_TIME_DIFF          6812 non-null   float64
5   AVERAGE_FREE_FLOW_TIME     6812 non-null   float64
6   LUMINOSITY                  6812 non-null   object
7   AVERAGE_TEMPERATURE        6812 non-null   float64
8   AVERAGE_ATMOSP_PRESSURE     6812 non-null   float64
9   AVERAGE_HUMIDITY           6812 non-null   float64
10  AVERAGE_WIND_SPEED         6812 non-null   float64
11  AVERAGE_CLOUDINESS         4130 non-null   object
12  AVERAGE_PRECIPITATION       6812 non-null   float64
13  AVERAGE_RAIN               563 non-null    object
dtypes: float64(8), object(6)
memory usage: 745.2+ KB
```

Figura 3: Missing Values dataset de Treino

Além disso, observou-se que alguns atributos exibem baixa variabilidade ou mesmo valores constantes ao longo de todo o *dataset*.

```
df_train["city_name"].unique()

array(['Porto'], dtype=object)
```

Figura 4: Dataset apenas para a cidade do Porto

```
df_train.describe()

#   AVERAGE_FREE_FLOW_TIME  AVERAGE_TEMPERATURE  AVERAGE_ATMOSP_PRESSURE  AVERAGE_HUMIDITY  AVERAGE_WIND_SPEED  AVERAGE_PRECIPITATION
0   6812.000000          6812.000000          6812.000000          6812.000000          6812.000000          6812.0
1    81.181952           16.192482           1017.388129          88.088190           3.028672          0.0
2    8.294401            5.163432            5.751061          18.238961           2.138471          0.0
3    86.302203            0.000000           985.000000          14.000000           0.000000          0.0
4    75.400000           13.000000          1015.000000          69.750000           1.000000          0.0
5    82.400000           16.000000          1017.000000          83.000000           3.000000          0.0
6    87.400000           19.000000          1021.000000          93.000000           4.000000          0.0
7   112.000000          35.000000          1033.000000          100.000000          14.000000          0.0
```

Figura 5: Valores da `average_precipitation`

A análise estatística efetuada nos *notebooks* também permitiu inspecionar os tipos de dados e a sua distribuição. Observou-se que todas as variáveis numéricas apresentam formatos compatíveis com operações estatísticas e de modelação ( `float64` ), enquanto as variáveis categóricas se encontram corretamente tipificadas como `object`. A exploração gráfica da distribuição das variáveis numéricas confirmou que muitas



delas seguem padrões pouco dispersos, sendo várias centradas em torno de poucos valores característicos, como acontece com `average_precipitation`.

Do ponto de vista temporal, a variável `record_date` encontra-se formatada de forma consistente nos dois *datasets*, possibilitando análises temporais ou, como foi feito, a criação de novas variáveis derivadas (como mês, dia da semana e horas de ponta).

### Distribuição de atributos relevantes

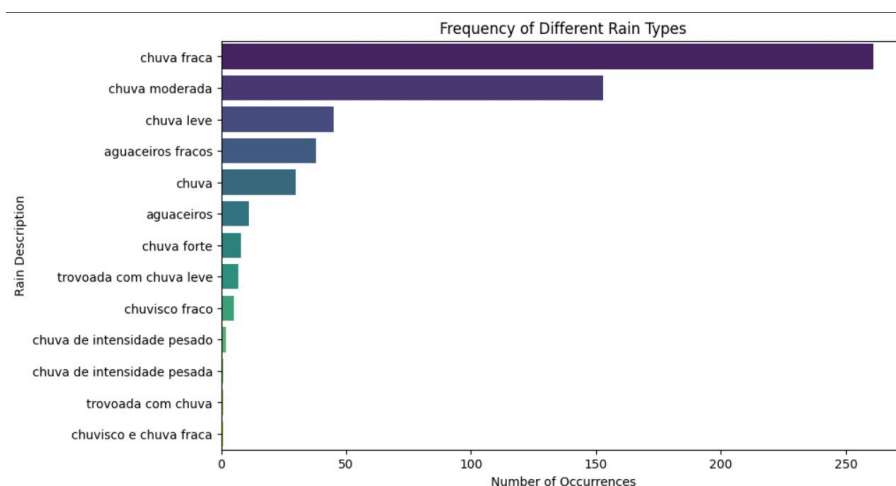


Figura 6: Distribuição dos tipos de chuva no *dataset* de treino ( `average_rain` )

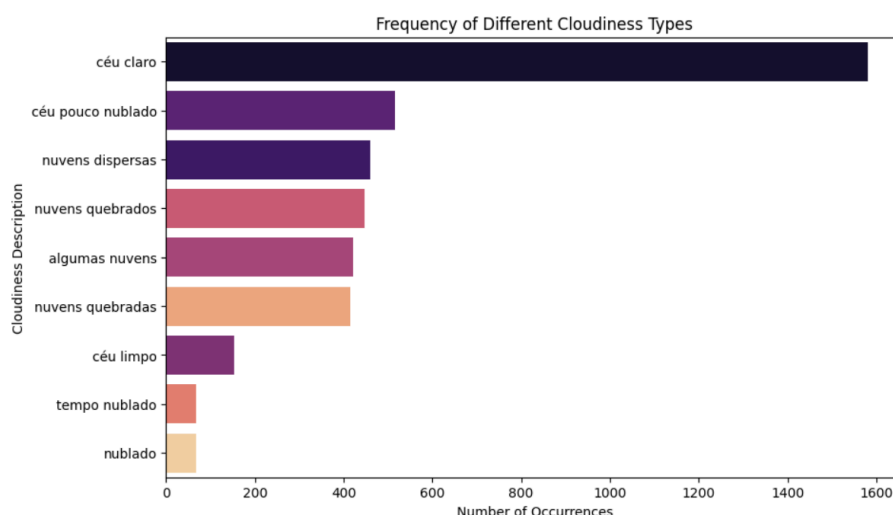


Figura 7: Distribuição dos tipos de nebulosidade no *dataset* de treino ( `average_cloudiness` )

As figuras geradas ao longo dos *notebooks*, incluindo gráficos de distribuição, histogramas e visualizações sobre *missing values*, tornaram evidente a necessidade de uma etapa rigorosa de preparação dos dados, especialmente no que diz respeito ao tratamento de valores em falta, *encoding* de categorias entre outros.

## 5. Data Preparation

Após a compreensão inicial dos dados e da identificação das suas limitações estruturais e semânticas, procedeu-se à fase de Data Preparation, onde foram aplicadas diferentes abordagens de limpeza, transformação e codificação dos atributos. Os tratamentos foram evoluindo progressivamente, refletindo um esforço contínuo para melhorar a qualidade dos dados e maximizar o seu valor para a modelação. Esta fase foi essencial para lidar com missing values, atributos redundantes, variáveis categóricas inconsistentes e elementos temporais ainda não explorados, garantindo que os modelos pudessem ser treinados sobre dados estáveis, limpos e representativos.

### 5.1. Primeira Abordagem

Na primeira abordagem, seguiu-se uma estratégia simples para obter rapidamente um conjunto de dados limpo e utilizável. Os *missing values* foram tratados através do preenchimento direto, tanto nas variáveis categóricas como na variável alvo, evitando a eliminação de linhas, visto que estes resultam de inconsistências no processo de parsing (importação) do ficheiro CSV, onde certos valores foram convertidos automaticamente para “NaN”. A coluna `record_date` foi inicialmente convertida para formato temporal, mas acabou por ser removida por não trazer valor imediato para a modelação e por exigir um tratamento mais aprofundado que seria explorado posteriormente.

Procedeu-se também a uma transformação significativa na coluna `average_cloudiness`, onde os valores originais foram reduzidos a duas categorias, sendo elas “ceu\_claro”, “pouco\_nublado” e “nublado” simplificando a variabilidade deste atributo. Em relação à `average_rain` aplicámos a mesma ideia e substituímos os *missing values* por “no\_rain”. As colunas `city_name` e `average_precipitation` foram eliminadas por apresentarem, respetivamente, valores constantes e informação praticamente nula.

Para concluir esta primeira fase, aplicou-se **Label Encoding** às variáveis categóricas e **Ordinal Encoding** à variável alvo, convertendo-as em valores numéricos sequenciais de forma simples, evitando o aumento da dimensionalidade.

### 5.2. Segunda Abordagem

Na segunda abordagem, foi adotada uma estratégia mais elaborada, focada principalmente na extração de informação temporal e na normalização semântica das variáveis categóricas. A coluna `record_date` foi reaproveitada e transformada em variáveis altamente informativas: **hora**, **dia da semana** e **mês**. Adicionalmente, foram criados indicadores binários para representar fins de semana e horas de ponta, características fortemente associadas aos padrões de tráfego urbano. Após esta decomposição, a coluna original de data já não era necessária e foi removida.

Para melhorar a qualidade dos dados meteorológicos, aplicou-se uma normalização de texto nas colunas `average_cloudiness` e `average_rain`, corrigindo problemas de codificação e consolidando categorias redundantes ou equivalentes. Os valores em falta foram substituídos por categorias explícitas, como “no\_rain” e “unknown\_cloudiness”, assegurando que essa ausência fosse tratada como informação válida e não como erro.

A coluna `city_name` foi novamente removida por conter apenas o valor constante “Porto”. A coluna `average_precipitation` foi igualmente excluída, uma vez que apresentava predominantemente valores nulos ou zero, contribuindo pouco para a variabilidade do modelo.

Finalmente, procedeu-se à transformação das variáveis categóricas `luminosity`, `average_cloudiness` e `average_rain` através de **One-Hot Encoding**, permitindo capturar diferenças entre categorias sem introduzir relações hierárquicas artificiais. Neste tratamento, foram também realizados testes com **Label Encoding** para comparação.

### 5.3. Terceira Abordagem

A terceira abordagem representou um refinamento adicional das estratégias anteriores. Mantiveram-se todas as operações efetuadas no Tratamento 2, nomeadamente extração temporal, normalização semântica, imputação explícita, remoção de atributos redundantes e codificação categórica, e adicionou-se uma análise cuidada de **outliers**.

A inspeção dos valores extremos permitiu verificar que estes não resultavam de erros de medição nem apresentavam valores incompatíveis com medições reais de tráfego ou meteorologia. Assim, optou-se por manter os **outliers**, reconhecendo que podem conter informação relevante sobre condições excecionais, como tempestades, horas de tráfego anormalmente intenso ou variações súbitas de velocidade.

### 5.4. Quarta Abordagem

Na quarta abordagem, a estratégia central focou-se na aplicação da técnica de **binning** para discretizar as variáveis numéricas contínuas, procurando reduzir o ruído e simplificar a variabilidade dos dados meteorológicos.

### 5.5. Quinta Abordagem

Tínhamos como objetivo realizar uma quinta abordagem, porém, apesar de ter sido iniciada, não tivemos tempo de concretizá-la. Tentamos expandir na abordagem 2, integrando dados externos que detalham a dinâmica social e de lazer da cidade do Porto para capturar fluxos de tráfego excecionais, nomeadamente os dias de jogos em casa do FC Porto no Estádio do Dragão e do Boavista no Estádio do Bessa, além das datas de realização do festival *Primavera Sound* desde 2018. Adicionalmente, integrou-se um calendário detalhado de feriados nacionais e locais, como o São João e o Carnaval, permitindo ao modelo distinguir dias com padrões de mobilidade urbana atípicos..

## 6. Modeling and Evaluation

Após a etapa de **Data Preparation**, procedemos à fase de **Modeling**, onde testamos e avaliamos vários algoritmos com o objetivo de prever a nossa variável alvo a partir dos atributos selecionados e transformados nas abordagens anteriores. Nesta fase, foram implementados sete modelos principais: **Decision Trees**, **Random Forest**, **XGBoost**, **k-Nearest Neighbors (kNN)**, **Stacking**, **Neural Networks (MLP Classifier)** e **Naive Bayes**, selecionados pela sua capacidade de lidar com dados tabulares, misto de variáveis categóricas e numéricas, e pela robustez que oferecem.

Para cada uma das versões do **Data Preparation** (Tratamento 1, Tratamento 2, Tratamento 3 e Tratamento 4), os modelos foram ajustados e avaliados, sendo posteriormente realizada uma fase adicional de otimização de hiperparâmetros utilizando **Grid Search**, aplicada na maioria dos tratamentos e modelos, mas com maior incidência nos modelos com o Tratamento 2. A definição dos parâmetros de pesquisa foi iterativa, testamos múltiplas combinações e amplitudes de parâmetros. No entanto, constatou-se que a expansão excessiva do espaço de busca resultava num aumento exponencial do custo computacional e tempo de treino sem que houvesse um ganho na métrica a melhorar. Para complementar, recorreu-se ao **Randomized Search** em cenários selecionados para explorar espaços com parâmetros mais vastos de forma mais rápida.

Adicionalmente, numa procura pelas otimizações ótimas, procedeu-se também à variação dos *random\_states* na maioria dos modelos. Testaram-se diversos valores para cada configuração, com o objetivo de identificar os valores que proporcionavam um melhor resultado.

### 6.1. Decision Trees

O primeiro modelo explorado foi o **Decision Tree Classifier**, pela sua simplicidade, interpretabilidade e baixo custo computacional. As árvores de decisão funcionam particionando o espaço de atributos em subconjuntos homogêneos, aprendendo regras claras de separação entre classes, o que permite compreender de forma intuitiva o comportamento do modelo.

Foram treinadas três versões do modelo correspondentes aos diferentes tratamentos de dados, desenvolvidos na fase anterior. Para o Tratamento 2 foi ainda aplicada uma versão otimizada através de **Grid Search**, que permitiu testar combinações de parâmetros como profundidade máxima, critério de divisão e número mínimo de amostras por nó.

**Resultados obtidos com o modelo Decision Trees:**



Figura 8: Melhores resultados do *Decision Trees* na plataforma Kaggle

### 6.2. Random Forest

O segundo modelo utilizado foi o **Random Forest Classifier**, um método de *ensemble* que se baseia na construção de múltiplas árvores de decisão, combinando os seus resultados por votação. Este tipo de modelo é particularmente eficaz em cenários com variabilidade significativa e dados ruidosos, sendo capaz

de capturar interações complexas entre variáveis e reduzir o risco de *overfitting* associado às árvores individuais.

À semelhança do modelo anterior, foram treinadas versões associadas a cada tratamento, bem como uma versão otimizada por **Grid Search** no Tratamento 2, onde se ajustaram parâmetros como o número de estimadores, profundidade das árvores e número de características consideradas por *split*.

**Resultados obtidos pelo modelo *Random Forest*:**



Figura 9: Melhores resultados do *Random Forest* na plataforma *Kaggle*

## 6.3. XGBoost

O terceiro modelo implementado foi o **XGBoost** (*Extreme Gradient Boosting*), reconhecido pela sua eficiência, robustez e elevado desempenho em dados tabulares. Este modelo baseia-se num processo iterativo de *boosting*, onde múltiplas árvores de decisão são construídas sequencialmente, com cada árvore corrigindo os erros da anterior.

Foi treinado com os diferentes tratamentos e, tal como nos restantes modelos, também se aplicou **Grid Search** e **Randomized Search** ao Tratamento 2 para otimização dos hiperparâmetros mais influentes.

**Resultados obtidos pelo modelo XGBoost:**



Figura 10: Melhores resultados do XGBoost na plataforma *Kaggle*

## 6.4. kNN

O modelo *k-Nearest Neighbors*(kNN) foi implementado como uma abordagem baseada em instâncias, onde a classificação de um novo ponto de dados é determinada pela classe maioritária dos seus “k” vizinhos mais próximos no espaço de atributos. Este algoritmo é sensível à escala dos dados e à métrica de distância utilizada, sendo útil para capturar padrões locais que modelos globais podem ignorar.

Para otimizar o desempenho do kNN, foi aplicado o **Grid Search** no conjunto de dados do Tratamento 2, explorando diferentes valores para o número de vizinhos (*n\_neighbors*), diferentes funções de peso (*weights*), como *uniform* ou *distance*, e métricas de distância como a *manhattan* e *euclidean*.

**Resultados obtidos pelo modelo kNN:**



Figura 11: Melhores resultados do kNN na plataforma *Kaggle*

## 6.5. Stacking

A técnica de *Stacking* foi introduzida como uma estratégia de *ensemble* avançada que procura superar as limitações de modelos individuais ao combinar as suas previsões através de um meta-modelo. Neste caso, utilizou-se o **StackingClassifier**, onde modelos base como **Random Forest**, **XGBoost** e **Decision Trees** geram previsões que servem de entrada para um classificador final, tipicamente uma **Regressão Logística**.

Esta abordagem permite ao modelo aprender qual o classificador base que é mais fiável para diferentes tipos de dados, resultando frequentemente numa maior capacidade de generalização. O processo foi validado através de **cross-validation** interna para garantir a robustez das previsões combinadas.

#### Resultados obtidos pelo modelo *Stacking*:

|                                                                                   |                |         |         |                          |
|-----------------------------------------------------------------------------------|----------------|---------|---------|--------------------------|
|  | stack2grid.csv | 0.81047 | 0.81777 | <input type="checkbox"/> |
| <small>Complete · Edgar Ferreira · 15d ago · Stack extended 2 grid</small>        |                |         |         |                          |

Figura 12: Melhores resultados do *Stacking* na plataforma *Kaggle*

## 6.6. Neural Networks (MLP Classifier)

As Redes Neurais foram exploradas através do algoritmo **Multi-layer Perceptron (MLP)**, um modelo de aprendizagem profunda capaz de modelar relações não-lineares complexas através de camadas de neurónios interligadas. O MLP utiliza o algoritmo de *backpropagation* para ajustar os pesos das ligações, minimizando o erro de classificação ao longo de várias épocas de treino.

A fase de modelação incluiu uma otimização rigorosa por **Grid Search** focada na arquitetura da rede, testando diferentes configurações de camadas ocultas (*hidden\_layer\_sizes*), funções de ativação como **tanh** e **relu**, e diferentes taxas de aprendizagem. Esta afinação permitiu encontrar o equilíbrio ideal entre a complexidade do modelo e o tempo de computação.

#### Resultados obtidos pelo modelo *Neural Networks*:

|                                                                                         |              |         |         |                          |
|-----------------------------------------------------------------------------------------|--------------|---------|---------|--------------------------|
|      | mlp2grid.csv | 0.78857 | 0.81111 | <input type="checkbox"/> |
| <small>Complete · Pedro Teixeira · 11d ago · MLP com Tratamento 2 e Grid Search</small> |              |         |         |                          |

Figura 13: Melhores resultados das *Neural Networks* na plataforma *Kaggle*

## 6.7. Naive Bayes

Finalmente, foi implementado o modelo **Gaussian Naive Bayes**, que se baseia na aplicação do Teorema de Bayes com a premissa de independência condicional entre todos os pares de atributos. Apesar da sua simplicidade, este modelo é extremamente rápido e eficaz, servindo frequentemente como uma excelente linha de base (*baseline*) para comparação com algoritmos mais complexos.

Por ser um modelo probabilístico que assume uma distribuição normal para os atributos numéricos, o GaussianNB não requer uma afinação extensiva de hiperparâmetros, sendo aplicado diretamente aos dados tratados para avaliar a sua performance em cenários de classificação rápida.

#### Resultados obtidos pelo modelo *Naive Bayes*:

|                                                                                     |             |         |         |                          |
|-------------------------------------------------------------------------------------|-------------|---------|---------|--------------------------|
|  | nb2grid.csv | 0.66571 | 0.64444 | <input type="checkbox"/> |
| <small>Complete (after deadline) · Pedro Teixeira · 16s ago</small>                 |             |         |         |                          |

Figura 14: Melhores resultados do *Naive Bayes* na plataforma *Kaggle*

## 6.8. Comparação de Resultados entre Modelos

A comparação dos desempenhos obtidos pelos três modelos revela várias tendências importantes. No caso das **Decision Trees**, observou-se uma melhoria gradual ao longo dos diferentes tratamentos, de 0.71333 no Tratamento 1 para 0.73111 no Tratamento 3, sendo o avanço mais expressivo alcançado com a aplicação de **Grid Search**, que elevou o desempenho para **0.80444**, demonstrando a forte dependência deste modelo em relação à otimização de **hiperparâmetros**. Já o **Random Forest** apresentou resultados

consistentemente superiores aos das árvores de decisão, atingindo **0.82** no Tratamento 2, o Tratamento 3 registou uma ligeira descida e a versão otimizada com **Grid Search** obteve o melhor resultado global de todo o estudo (**0.83555**). Por fim, o **XGBoost** mostrou-se um modelo estável e competitivo, com melhorias modestas entre os tratamentos e alcançando 0.80888 após otimização, posicionando-se como uma alternativa robusta embora ligeiramente inferior ao **Random Forest** otimizado.

Em síntese, o **Random Forest com Grid Search** destaca-se como o modelo mais eficaz nesta fase de modelação, o XGBoost surge como uma solução consistente e de elevado desempenho e as **Decision Trees**, apesar da simplicidade, mostram claras melhorias com ajustamentos mais avançados, servindo como uma boa referência base.

## 7. Conclusões

Ao concluir este trabalho sobre a previsão de tráfego urbano na cidade do Porto, é possível afirmar que os objetivos propostos foram alcançados com sucesso. A aplicação sistemática de técnicas de Aprendizagem Automática permitiu desenvolver modelos capazes de prever a variável `average_speed_diff` com um nível de desempenho competitivo no contexto da competição *Kaggle*.

A exploração de diferentes estratégias de preparação de dados revelou-se determinante para a qualidade das previsões. Em particular, a engenharia de atributos temporais, a normalização semântica das variáveis categóricas e o tratamento explícito de valores em falta demonstraram ter um impacto significativo no desempenho dos modelos, confirmando a importância crítica da fase de *Data Preparation* em projetos de Machine Learning.

Do ponto de vista da modelação, a comparação entre vários algoritmos evidenciou a robustez dos métodos de ensemble. O modelo *Random Forest*, após otimização de hiperparâmetros através de *Grid Search*, destacou-se como a solução com melhor desempenho global, atingindo um valor máximo de 0.83555. Estes resultados reforçam a ideia de que a afinação criteriosa de parâmetros é tão relevante quanto a escolha do algoritmo em si.

Apesar dos resultados obtidos, reconhecemos que existem oportunidades de melhoria. Como trabalho futuro, seria pertinente explorar técnicas mais avançadas de seleção de atributos e integrar fontes de dados adicionais, como informação em tempo real sobre acidentes, obras rodoviárias ou eventos urbanos, que possam influenciar padrões de tráfego de forma súbita. Adicionalmente, a experimentação com outras arquiteturas e abordagens poderia contribuir para um aprofundamento ainda maior da análise.

Em síntese, este projeto permitiu consolidar conhecimentos teóricos e práticos em Aprendizagem Automática, evidenciando a complexidade e o potencial destas técnicas quando aplicadas a problemas reais de mobilidade urbana.



## **8. Avaliação por Pares**

- Edgar - 0
- Fernando - 0
- Pedro - 0
- Sara - 0